Digital Systems

Lecture 14

Context Switching

Mark van der Wilk

Hilary Term 2026

Department of
COMPUTER
SCIENCE

Following the course of past years by Mike Spivey

Today

How does the OS switch tasks?

- How does it decide what to run next?
- How does it move messages?
- **How does it actually do the context switch?**

Context Switching

Switch processes on `send()`, `receive()`, `yield()`, or interrupt.

Temporarily passes control to OS, which then decides what to do next.

The plan to do so:

- Enter the OS via a software interrupt instruction `svc`, or by a normal interrupt.
- Save *entire* processor state on the stack.
- After choosing a new process, restore its state to continue.

Operating System is basically one big interrupt

`svc` interrupt vs normal interrupt

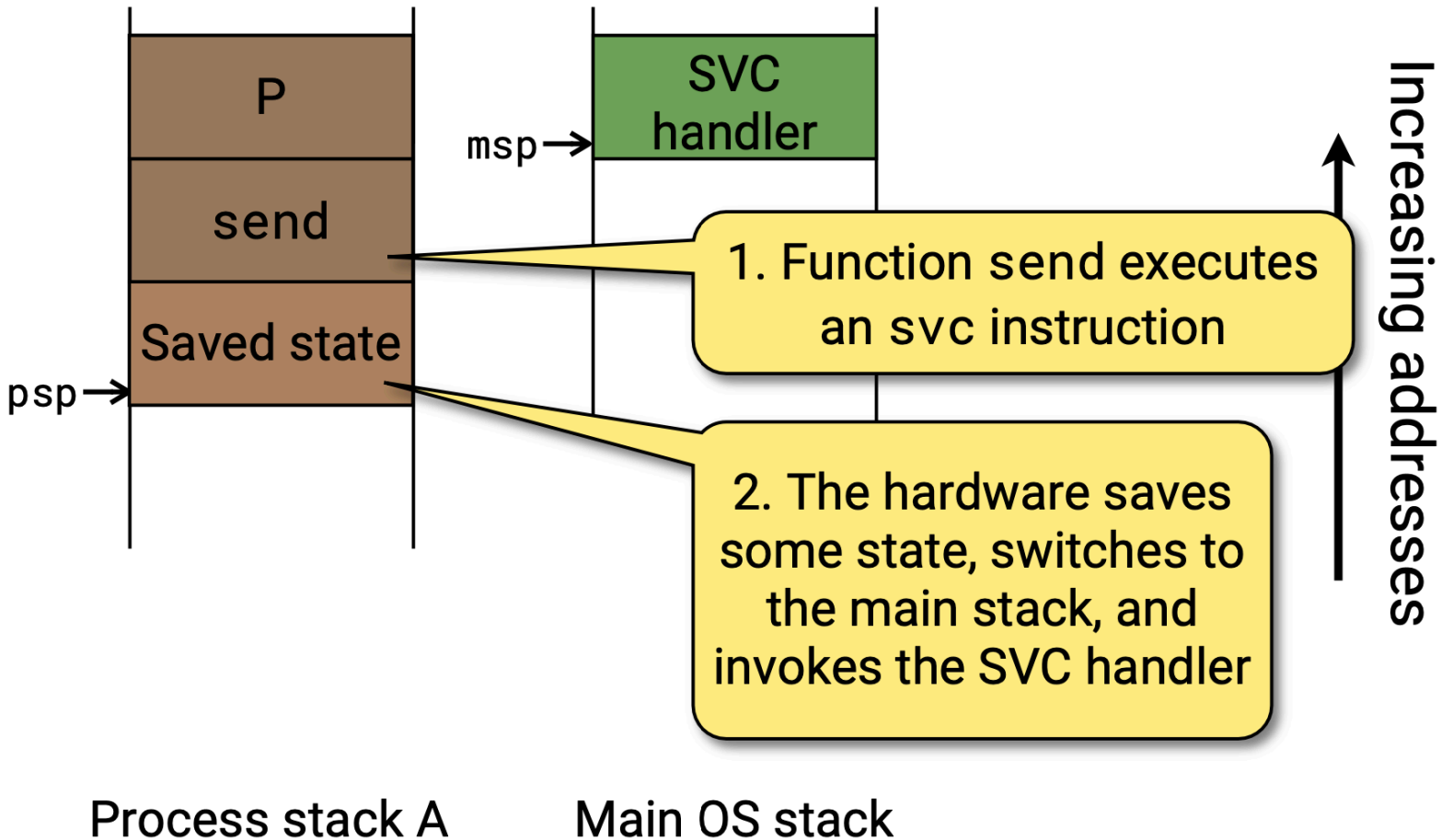
Instruction `svc` (supervisor call) operates like a normal interrupt:

- Hardware stores the usual state to conform to calling convention.
- `r0-r3`, `r12`, `lr`, `pc`, `psr`
- Magic value is placed in `lr`, so hardware knows to do special return.

Differences:

- Hardware moves CPU to different state (traced in `CONTROL`), from "user" to "kernel".
- `sp` now patches through a different register, so OS has its own stack.
- Different magic value is placed in `lr`.

Context Switch I



Context Switch II

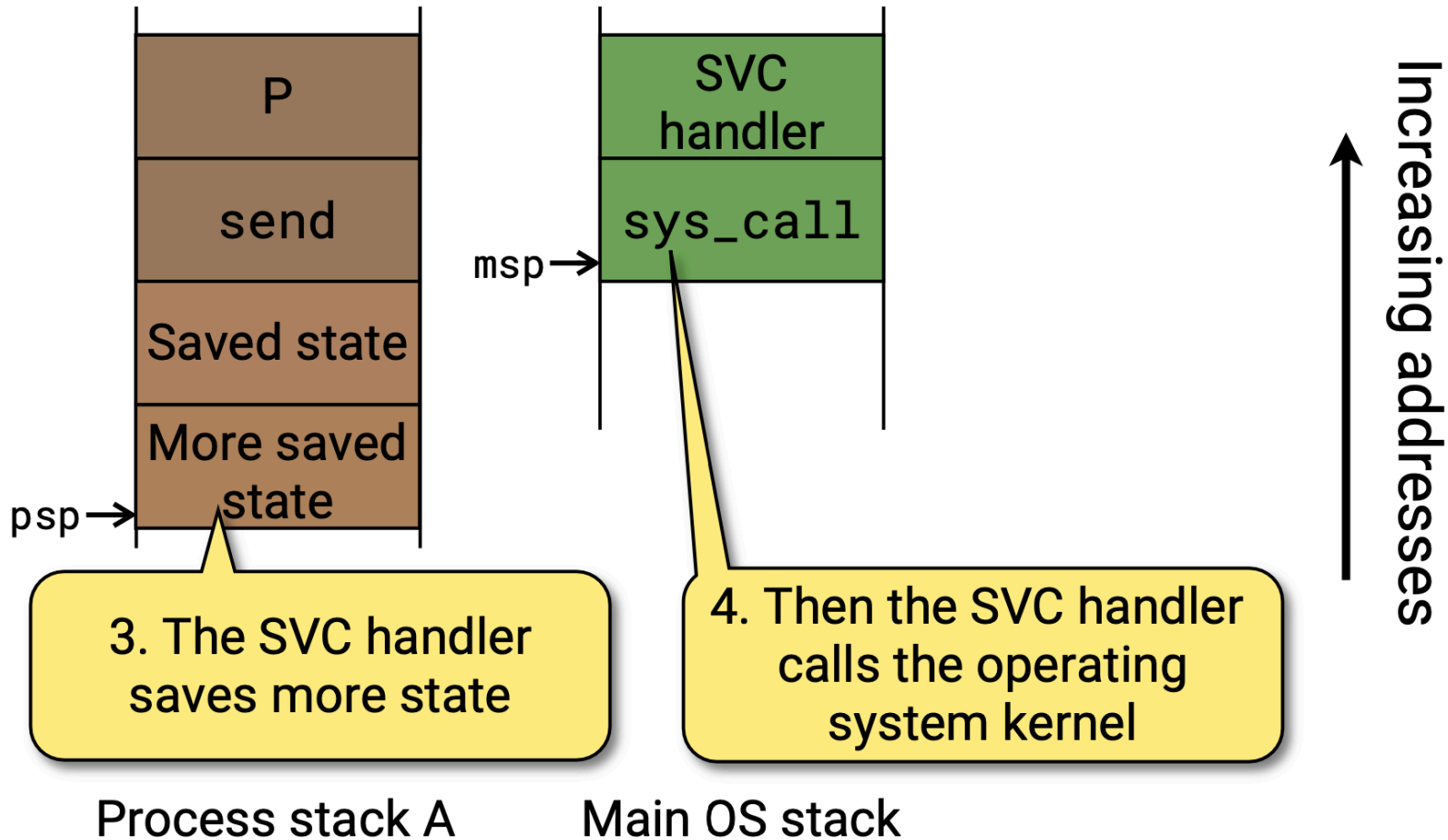
We additionally want to be able to run a different process!

? Difference between interrupt and context switch

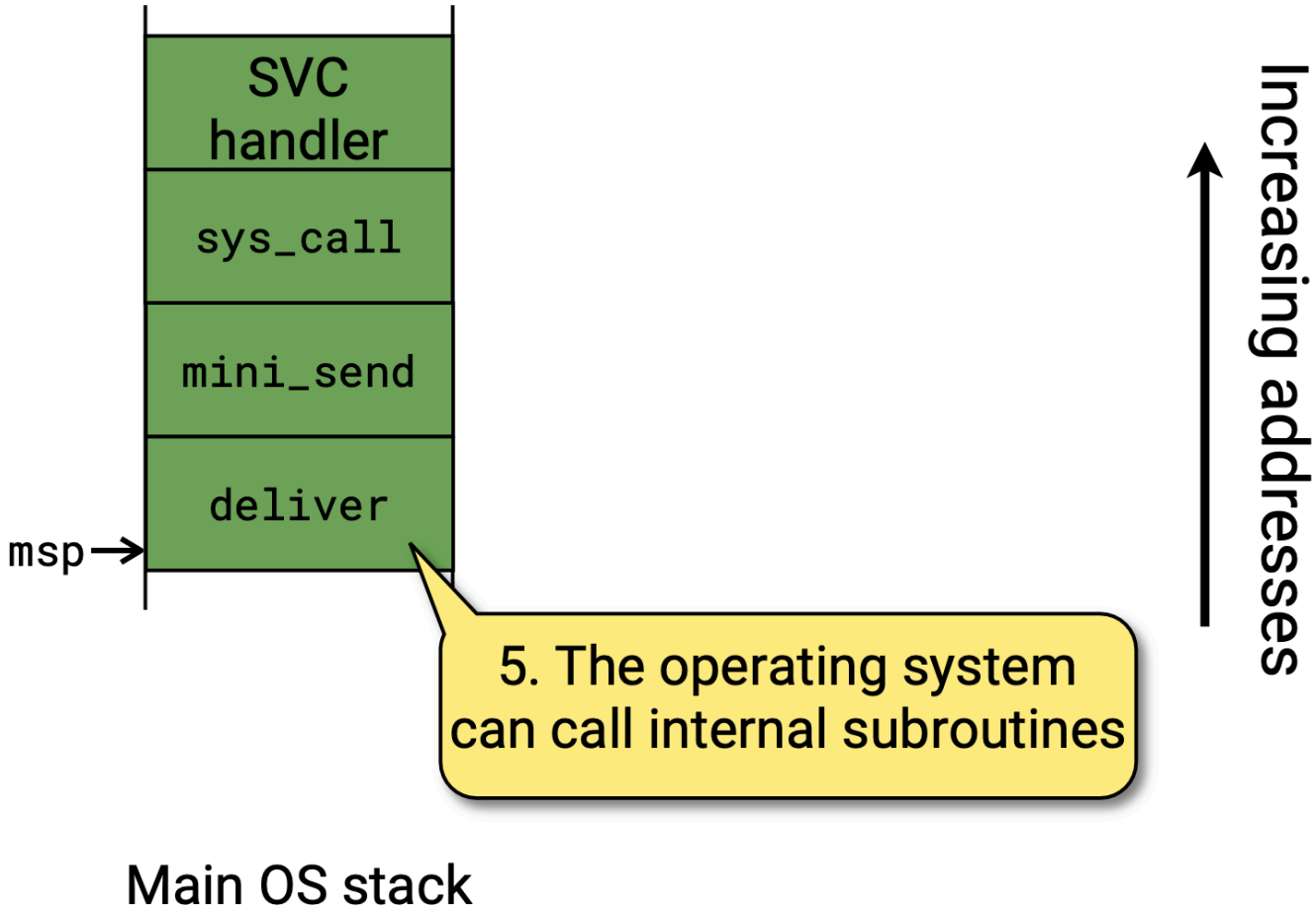
Q: What contract do the leaving/incoming processes have with each other? What is the difference to a normal interrupt? What is the difference to a normal subroutine call?

- We cannot rely on subroutine conventions to restore state in correct way,
- $\Rightarrow$ We must store *all* state of the process on the stack.

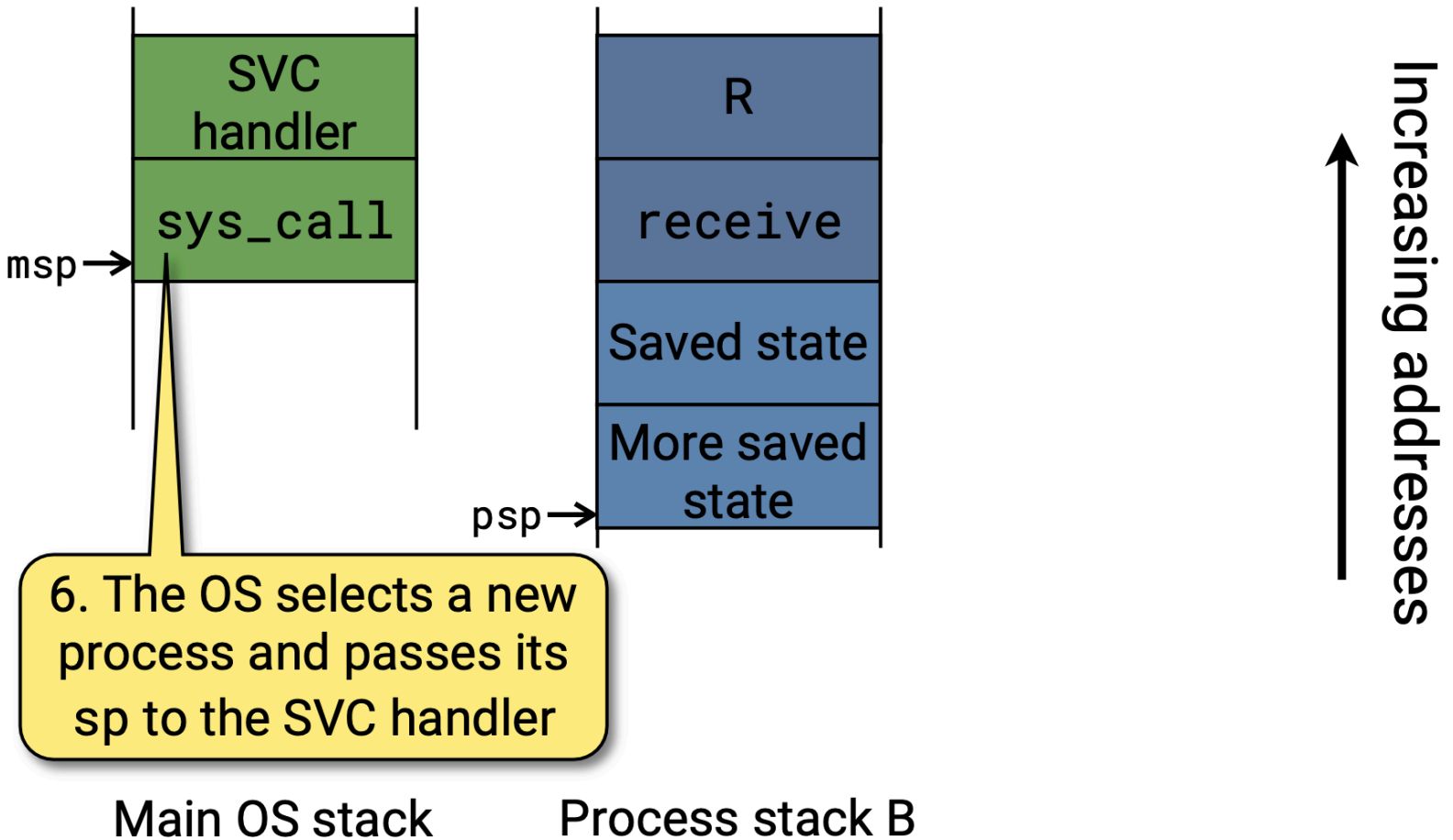
Context Switch II



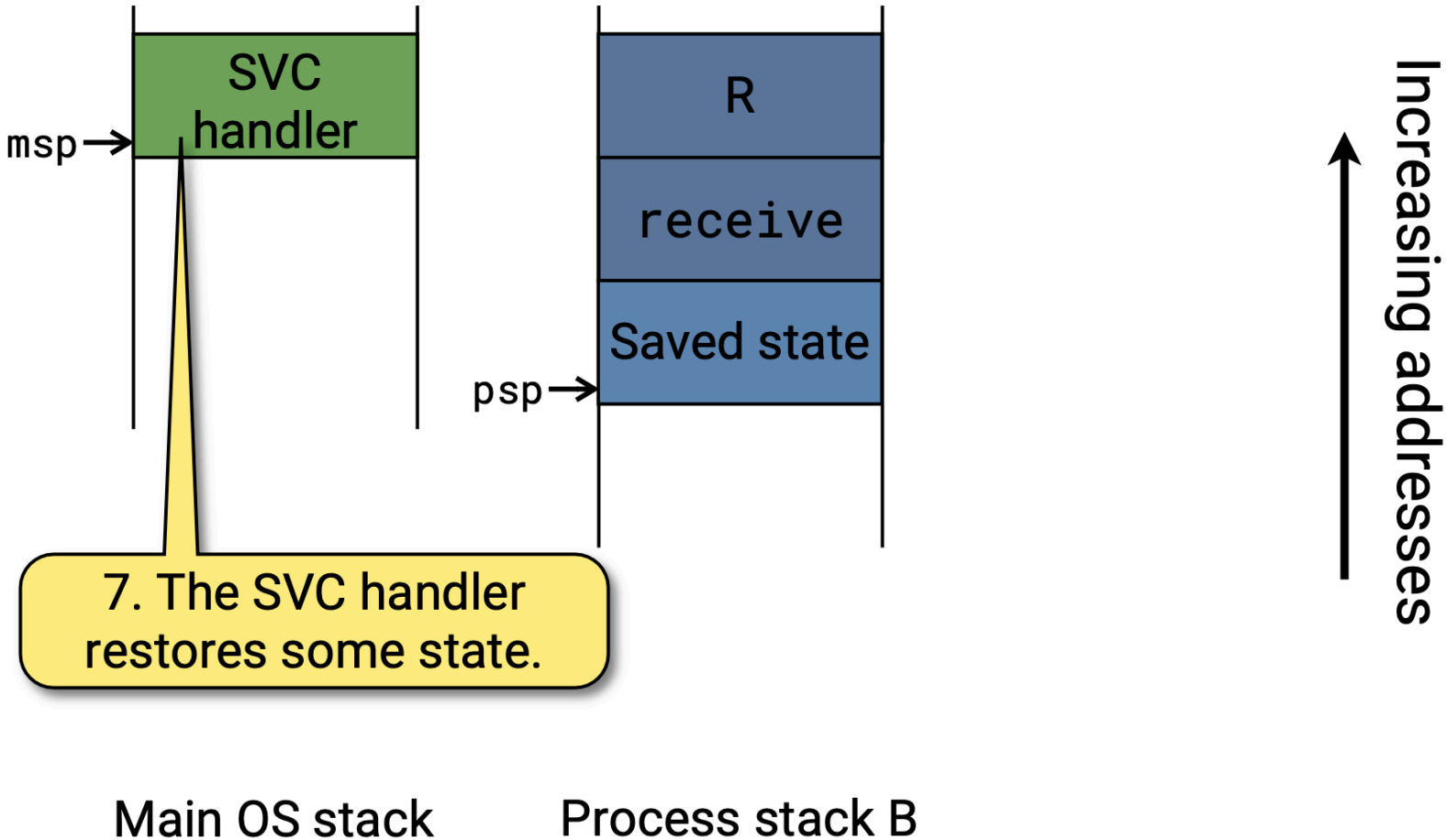
Context Switch III



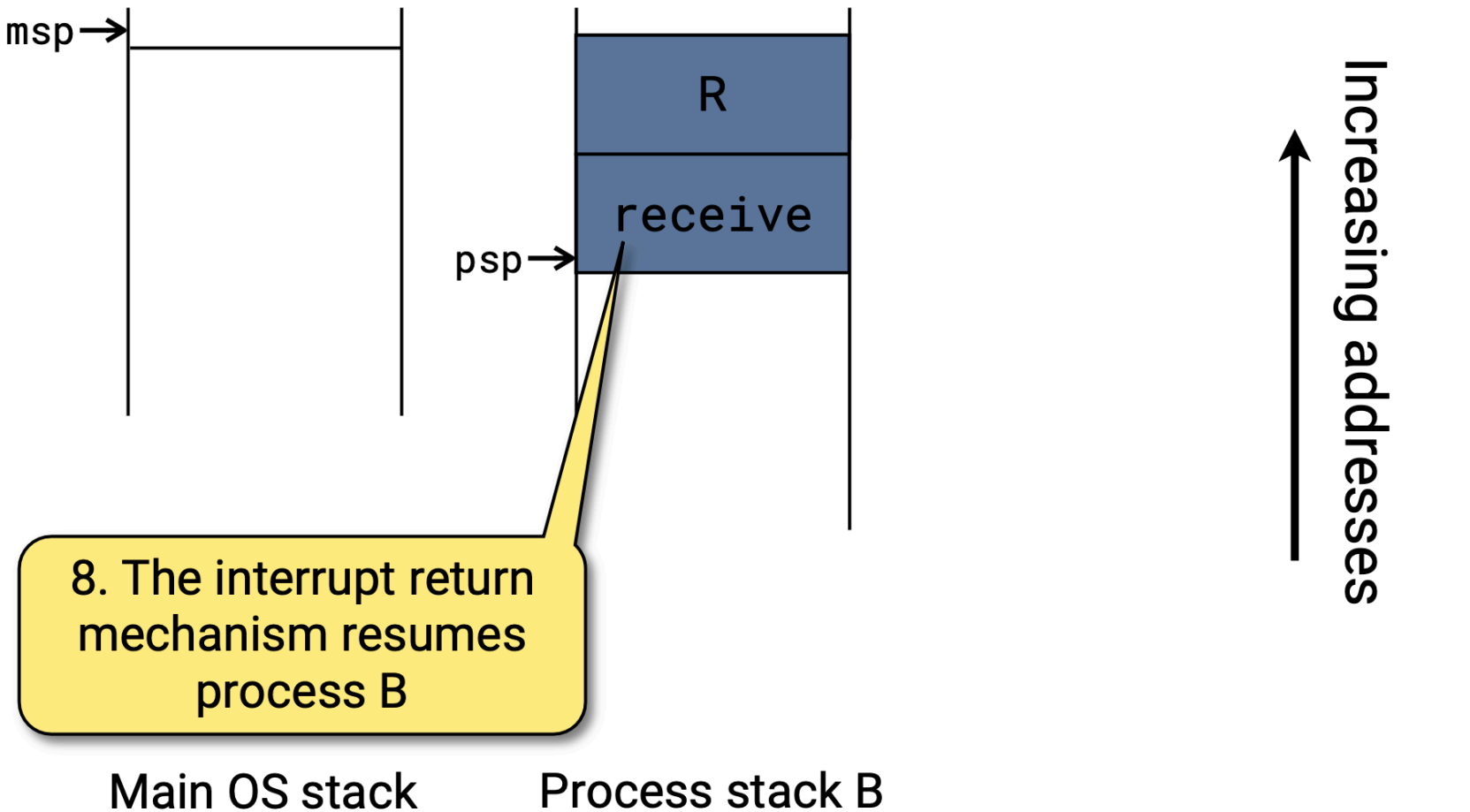
Context Switch IV



Context Switch V



Context Switch VI



Initiating System Calls

```
void NOINLINE yield(void) {
    syscall(SYS_YIELD);
}

void NOINLINE send(int dest, int type,
                   message *msg) {
    syscall(SYS_SEND);
}
```

OS will find arguments in registers `r0-r2`

Generates an `svc` instruction

Handling System Calls

Sequence of events:

- `startup.c`: Vectors table determines what gets called on syscall.
- `mpx-m0.s`: `svc_handler` stores remaining state.
- `microbian.c`: `system_call()` handles call, determines next process, returns stack pointer of process.
- `mpx-m0.s`: `svc_handler` restores state of (new) process.
- `bx ...`: Returns to magic number `0xfffffffd`

Where is `svc_handler`?

```
grep -rnw *.c *.s -e svc_handler
```

svc_handler

Navigating source files is hard! IDE can help with this, or can search for text using `grep`: `grep -rnw *.c *.s -e svc_handler`.

```
$ grep -rnw *.c *.s -e svc_handler
startup.c:142:void svc_handler(void);
startup.c:186:    svc_handler,
mpx-m0.s:73:@@@ svc_handler -- handler for SVC interrupt
(system call)
mpx-m0.s:74:    .global svc_handler
mpx-m0.s:76:svc_handler:
```

`svc_handler`

Navigating source files is hard! IDE can help with this, or can search for text using `grep`: `grep -rnw *.c *.s -e svc_handler`.

```
svc_handler:
    isave @ Complete saving of state
    @@ Argument in r0 is sp of old process
    bl system_call @ Perform system call
    @@ Result in r0 is sp of new process
    irestore @ Restore saved state
```

Saving the State

Unavoidable bit of assembly to store additional state, since this is not done by hardware, cannot be done by calling convention, and C cannot be trusted to get this right.

```
@@@ isave -- save context for system call
    .macro isave
    mrs r0, psp             @ Get thread stack pointer
    subs r0, #36
    movs r1, r0
    mov r3, lr              @ Preserve magic value
    stm r1!, {r3-r7}        @ Save low regs on thread stack
    mov r4, r8              @ Copy from high to low
```

```
mov r5, r9
mov r6, r10
mov r7, r11
stm r1!, {r4-r7}    @ Save high regs on thread stack
.endm               @ Return new thread sp
```

System Call: OS Side

```
unsigned *system_call(unsigned *psp) {
    short *pc = (short *) psp[PC_SAVE];
    int op = pc[-1] & 0xff;
    os_current->sp = psp;
    switch (op) {
    case SYS_YIELD:
        make_ready(os_current);
        choose_proc();
        break; ...
    }
    return os_current->sp;
}
```

Remaining details

- How to start a process.
- How to start the entire operating system.
- How to schedule processes (next time).